

- [PAYOO SDK FOR MOBILE APPLICATION]

PAYOO PAYBILL SDK INTEGRATION OVERVIEW

CHANGE LOG

Date	Version	Author	Approver	Description
15/08/2019	1.0.0	Nhan Nguyen	Anh Le	Created first version
03/09/2019	2.0.0	Nhan Nguyen	Anh Le	Update Android document
04/09/2019	2.0.1	Nhan Nguyen	Anh Le	Update iOS document
09/12/2019	2.1.0	Dai Tran	Nhan Nguyen	Update iOS document
10/01/2020	2.1.1	Huy Nguyen	Nhan Nguyen	Update Android document
08/03/2021	2.1.2	Huy Nguyen	Nhan Nguyen	Update Bill payment flow

TABLE OF CONTENTS

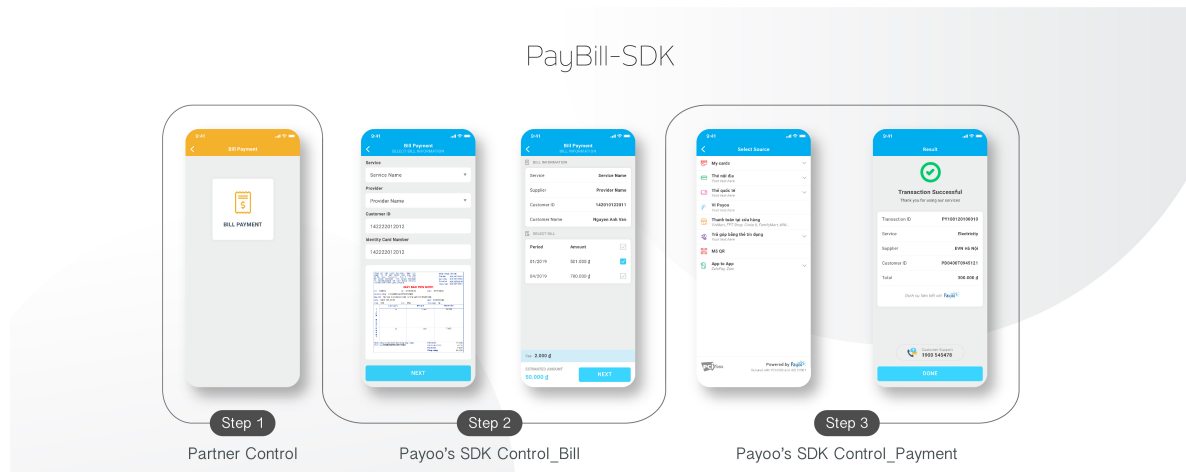
- [Introduction](#)
- [Configuration](#)
- [Integration](#)
 - [iOS](#)
 - [Android](#)
- [Appendix](#)
 - [API/ VietUnion Function](#)
 - [Security and Encryption](#)
 - [SSL/HTTPS](#)
 - [Static IP Addresses](#)
 - [API Authentication Credentials](#)
 - [Digital Signature Authentication](#)
 - [API/ VietUnion Function](#)
 - [API General Structure](#)
 - [API Online Bill Payment](#)
 - [API Get Transaction Status](#)
- [Contact](#)

Aiming to become the leading intermediary payment service provider in Vietnam, **VietUnion** introduces **Payoo**, a payment solution that satisfies the demands of the flexible time of payment and payment method for every customer.

VietUnion focuses on improving and providing the safest and most convenient payment solution in order to give customers the best benefits.

Introduction

With Payment gateway



Bill payment using Payment gateway UI Flow

Step #1. Partner App calls the SDK. Paybill payment process happens in the SDK.

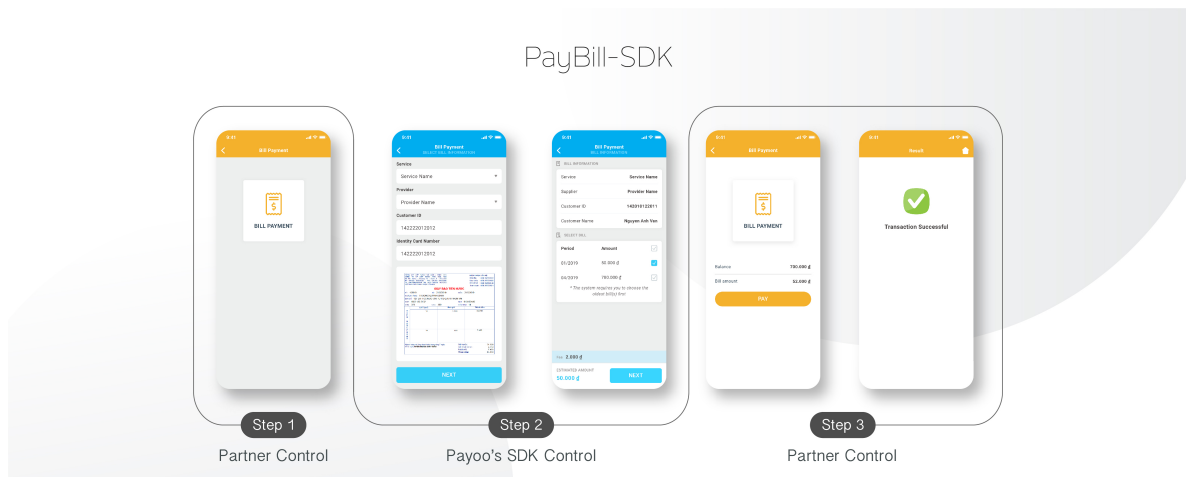
Step #2.

- User chooses service, provider and input customer code or additional required fields that need to query.
- User selects bill that need to pay based on month/period or specific range.
- User chooses available payment method (Ex: Domestic/International Card)
- User fills in required information.
- User confirms to finish payment.

*** If OTP is required, user should follow bank instructions to complete the payment process.**

Step #3. SDK returns result to Partner App.

Without payment gateway



Bill payment without payment gateway UI Flow

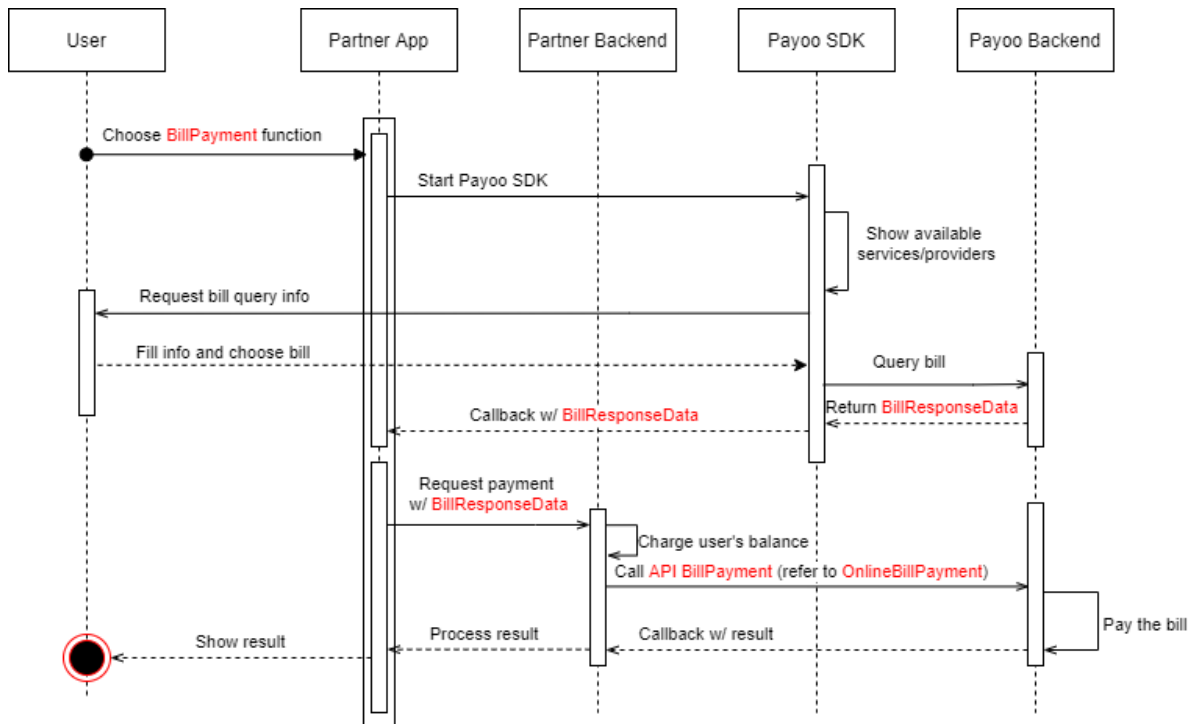
Step #1. Partner App calls the SDK. Paybill payment process happens in the SDK.

Step #2.

- User chooses service, provider and input customer code or additional required fields that need to query.
- User selects bill that need to pay based on month/period or specific range.

Step #3.

- SDK returns bill result to Partner App.
- Partner Backend charges user's balance.
- Partner Backend calls Payoo Backend to pay the bill.



Bill payment without payment gateway Flow Diagram

Configuration

- **MerchantID** : this configuration parameter will be changed in production system.
- **MerchantShareKey** : this configuration parameter will be changed in production system.

Integration

iOS

Prerequisites

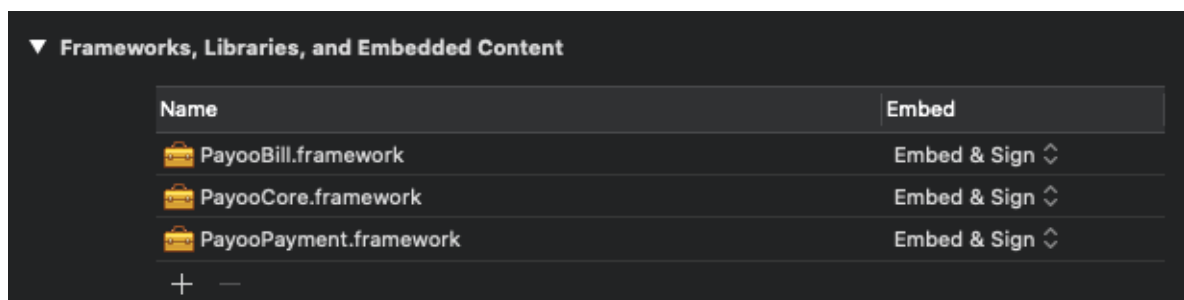
- Xcode: 9.0 or later.
- iOS: 9.0 or later.

Installation

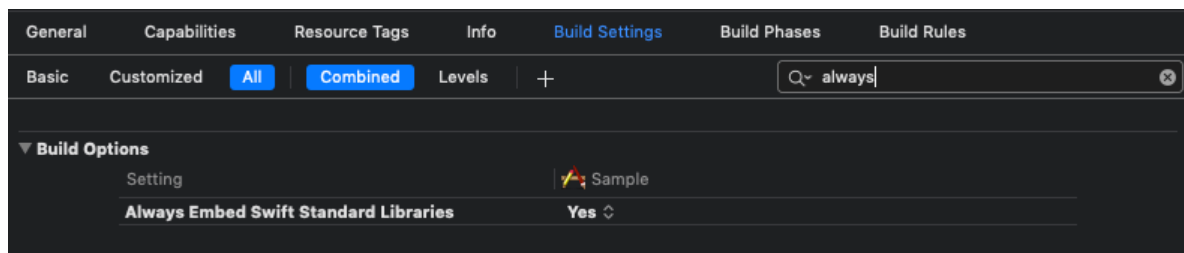
#Step 1. Open your project in Xcode, and drag the **PayooPayment.framework**, **PayooBill.framework** and **PayooCore.framework** into the Project Navigator of your application's Xcode project.

They should appear nested underneath your application's blue project icon. Whether they are above or below all the other Xcode groups do not matter.

#Step 2. Select your application project in the Project Navigator (blue project icon) to navigate to the target configuration window and select the application target under the **"Targets"** heading in the sidebar. On the tab bar at the top of that window, open the **"General"** panel. Click on the **+** button under the **"Embedded Binaries"** section. Select **PayooPayment.framework**, **PayooBill.framework** and **PayooCore.framework**.



#Step 3. [For Objective-C project] Select the **"Build Settings"** panel on the tab bar of the project editor, search for **Always Embed Swift Standard Libraries** and set its value to Yes.



#Step 4. Select the **"Build Settings"** panel on the tab bar of the project editor, search for **Import Paths**, add value **\$(SRCROOT)** and set it to recursive.

#Step 5. (Only Xcode 9.4.1 or below) Create a file module.map in your project directory (no need to add it to Xcode) with the following code:

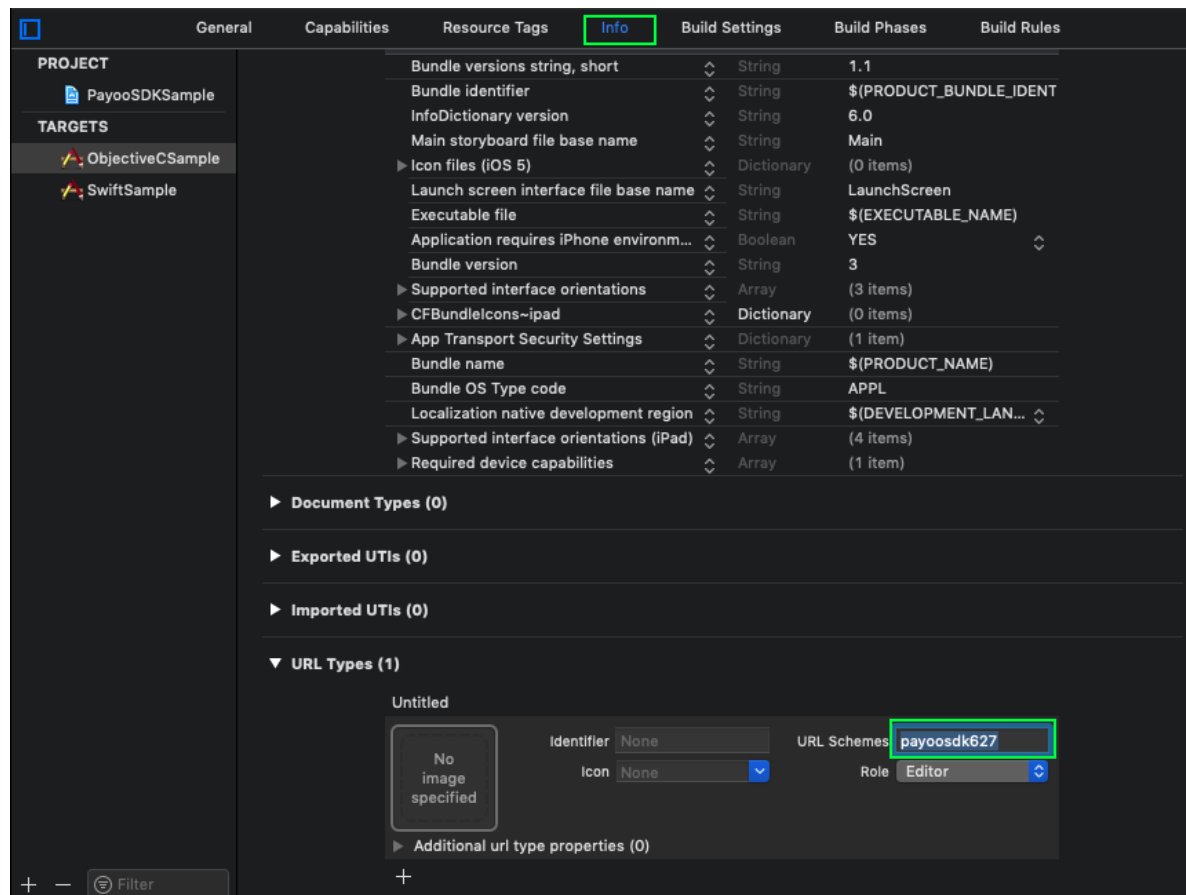
```

module CommonCrypto [system] {
    header
    "/Applications/Xcode.app/Contents/Developer/Platforms/iPhoneOS.platform/Developer/SDKs/iPhoneOS.sdk/usr/include/CommonCrypto/CommonCrypto.h"
    export \*
}

```

#Step 6. Select the **"Info"** panel on the tab bar of the project editor, expand **URL Types** header and set **"URL Schemes"** value to the structure: **"payoosdk" + provided merchant ID**.

For example, if your merchant ID is 627__, your "URL Scheme" has to be payoosdk627.



#Step 7. In Info.plist, append the following XML code:

```

<key>NSAppTransportSecurity</key>
<dict>
    <key>NSAllowsArbitraryLoads</key>
    <true/>
</dict>

```

#Step 8. Setup authorization information first by setting provided merchant ID and merchant secret key to the SDK.

```

@objc(PayooBillConfiguration)
public final class BillConfiguration: NSObject {
    public static func set(merchantId: String , secretKey: String )
}
// sample code
BillConfiguration.set(merchantId: merchantId, secretKey: secretKey)

```

#Step 9. Implement following method in AppDelegate class:

```
func application(_ app: UIApplication , open url: URL , options:
[UIApplication. OpenURLOptionsKey : Any] = [:]) -> Bool {
    return PaymentAppDelegate.shared.application(app, open: url,
options: options)
}
```

#Step 10. Select **Build Phases** panel on the tab bar at the project editor. Click the + button at the upper-left corner of the Build Phases editor and select **New Run Script Phase**. In the script editor box, paste the following script:

```
bash
"${BUILT_PRODUCTS_DIR}/${FRAMEWORKS_FOLDER_PATH}/PayooCore.framework/strip-
frameworks.sh"
```

Usage

Create Bill Configuration

```
let billConfiguration = BillConfiguration (
    defaultServiceId: String,
    defaultProviderId: String,
    defaultCustomerId: NSNumber,
    referAmount: NSNumber?,
    userId: String?,
    defaultCustomerEmail: String?,
    defaultCustomerPhone: String?
)
```

Create Bill Context

```
let billContext = BillContext (
    configuration: BillConfiguration,
    completionHandler: BillCompletionHandler?
)
```

Sample code:

```
let billContext = BillContext (configuration: billConfig) { status, data in
    switch status {
    case .success:
        self.showAlert(title: "Success", message: "\(data!.message) (\(
data!.code))")
    case .failure:
        self.showAlert(title: "Failure", message: "\(data!.message) (\(
data!.code))")
    case .unknown:
        self.showAlert(title: "Unknown", message: "\(data!.message) (\(
data!.code))")
    case .cancel:
        self.showAlert(title: "Cancel", message: "Cancelled by user.")
    }
}
```

GroupType:

```
public enum GroupType {  
    case success  
    case failure  
    case unknown  
    case cancel  
}
```

Data:

```
public class BillResponseObject {  
  
    public let code: Int // Mã trạng thái đơn hàng  
    public let message: String? // Mô tả code  
    public let data: BillResponseData? // Thông tin đơn hàng  
}  
  
public class BillResponseData {  
  
    public let orderId: String? // Mã đơn hàng  
    public let itemCode: String? // Mã dịch vụ đã được thanh toán  
    public let totalAmount: Double // Tổng tiền thanh toán  
    public let paymentFee: Double // Phí thanh toán  
    public let paymentCode: String?  
    public let orderInfo: String?  
    public let orderChecksum: String?  
    public let authToken: String?  
}
```

Customization

- The background color of navigation bar

```
Appearance.navigationBarBackgroundColor = UIColor.white
```

- The background color of the Paybill SDK

```
Appearance.backgroundColor = UIColor.white
```

- The tintcolor of navigation bar

```
Appearance.navigationBarTintColor = UIColor.white
```

- The background color of continue button

```
Appearance.continueButtonColor = UIColor.white
```

- The status bar style

```
Appearance.preferredStatusBarStyle = .lightContent
```

- Running environment The SDK supports production and development environment. The default running environment is production.

```
enum Environment {  
    case production  
    case development  
}  
@objc(PayooConfiguration) public final class Configuration: NSObject {  
    public static func set (environment: Environment)  
}
```

- UI language The SDK supports Vietnamese and English. The default language is Vietnamese.

```
public enum Language {  
    case vietnamese  
    case english  
}  
@objc(PayooConfiguration) public final class Configuration: NSObject {  
    public static func set(language: Language)  
}
```

Android

Prerequisites

- minSdkVersion 15
- buildTools 3+
- Android Studio 3+

Installation

Step #1. Copy these files into **app/libs**

- payoo-core-release-<x.y.z>.aar
- payoo-paybill-sdk-<x.y.z>.aar
- payoo-payment-sdk-<x.y.z>.aar

Step #2. Add following lines into your gradles:

- **build.gradle (project):** under **buildscript**

```
ext.kotlin_version = '1.3.72'
```

Step #3. In **build.gradle (app)** file, add these lines at top level:

```
apply plugin: 'kotlin-android'  
apply plugin: 'kotlin-kapt'
```

Step #4. In **build.gradle (app)** file, add these lines into dependencies node:

```
// x.y.z is the version of sdk
implementation(name: 'payoo-core-release-<x.y.z>' , ext: 'aar') { transitive =
    **true** }
implementation(name: 'payoo-paybill-sdk-<x.y.z>' , ext: 'aar') { transitive =
    **true** }
implementation(name: 'payoo-payment-sdk-<x.y.z>' , ext: 'aar') { transitive =
    **true** }
implementation"org.jetbrains.kotlin:kotlin-stdlib-jdk7:$kotlin_version"
```

Usage

Initialize SDK

Step #1. Create PayooMerchant instance (**vn.payoo.paybillsdk**)

```
PayooMerchant payooMerchant = PayooMerchant.newBuilder()
    .merchantId(<YOUR_MERCHANT_ID>)
    .secretKey(<YOUR_SECRET_KEY>)
    .environment(<PAYBILL_ENVIRONMENT>) // refer to PaybillEnvironment.class
    .build();
```

Step #2. Init Payoo Paybill SDK in your application

Add this lines into your **Application** level class (ex: App.java)

```
PayooPaybillSDK.initialize(this, payooMerchant);
```

Call payment method with your order object

Step #1. Create **PaybillConfig** instance

```
PaybillConfig paybillConfig = PaybillConfig.newBuilder()
    .withLocale(<YOUR_CURRENT_LOCALE>) // used to render UI based on language

    .withStyleResId(<YOUR_CUSTOM_THEME_ID>) // used to apply custom theme
    color. Refer to section 4. Customization.
    .withCustomerEmail(YOUR\_DESIRED\_EMAIL) // used to receive payment
    confirmation. Omit this if you wish to input manually.
    .withCustomerPhone(<YOUR_DESIRED_PHONE_NUMBER>) // used for auto fill.
    Omit this if you wish to input manually.
    .withServiceId(<YOUR_DESIRED_SERVICE_ID>) // used to filter specific
    Service. Omit this if you wish to select manually
    .withProviderId(<YOUR_DESIRED_PROVIDER_ID>) // used to filter specific
    Provider from selected Service. Has no effect alone.
    .withCustomerCode(<YOUR_DESIRED_CUSTOMER_CODE>) // used for auto fill.
    Must pass in along .withServiceId() & .withProviderId() to take effect. Omit
    this if you wish to input manually.
    .withReferAmount(<TOTAL_MONEY_AMOUNT>) // used for auto fill
    .build();
```

Step #2. Call Paybill SDK

Add this line to call Paybill SDK inside an Activity or Fragment and handle purchasing result returned from SDK

```

PayooPaybillSDK.start( this , paybillConfig, new
PayooPaybillSDK.OnPaybillResultListener () {
    @Override
    public void onResult(@NonNull PaybillResultResponse data) {
        @GroupType int groupType = data.groupType // refer to Return code
        if (groupType == GroupType.SUCCESS) {
            // proceed with response data
        }
    }
});

```

Return code

```

GroupType //(vn.payoo.paybillsdk)
public @interface GroupType {
    int CANCEL = 2;
    int FAILURE = -1;
    int SUCCESS = 0;
    int UNKNOWN = 1;
}

```

Customization

- Apply custom colors to sdk in your **styles** resource (ex : **R.styles.PaybillSDKTheme**)
- Allow changing three primary colors in Android: **colorPrimary** , **colorPrimaryDark** and **colorAccent**
- **colorToolbar** and **colorHighlight** must be defined.
- Colors must come from **color** resources (ex: **R.color.colorSample**) to take effect, not hex codes (ex: **#A1EF000**)
- Refer to sample project for more information

Appendix

API/ VietUnion Function

Security, Encryption

These are security elements of UniGWS implement:

- SSL/HTTPS => data encode, anti-theft, and anti-forgery.
- Verify static IP Address => Check authorization to know if authorized Partner queries objects or not.
- Verify VietUnion Partner through API Credentials (RequestTime, AgentId) => Check authorization, anti-replay.
- Verify digital signature => authentication, data integrity, non-repudiation.

SSL/HTTPS

Purpose of using SSL/HTTPS is for data exchange between Backend Partner and VietUnion to be encoded. It's difficult for theft and forgery.

- SSL/HTTPS is applied on connectivity between UniGWS and Backend partners.

- As deployed, UniGWS webservice is setup SSL with a pair of private key/certificate of VietUnion.

Static IP Addresses

VietUnion needs to verify Static IP of Partner, one or more IPs depending on Partner requirements/ agreement between two parties. VietUnion will maintain Partner static IP list in the configuration File/ Database.

Upon receiving the requests, the API of UniGWS will verify IP by checking the static IP's list of Partner. If not belong to authorized Partner, UniGWS will refuse to executing queries.

API Authentication Credentials

Function is to verify that request sender provides IP/Password correctly which are issued before. Thereby, verifying the request is valid or not.

When connecting to any API of UniGWS, Backend Partner will send the following parameters:

Params	Description
AgentId	Identify for each Partner. AgentID is issued by VietUnion. For example: "5678".
RequestTime	Timestamp to call API (dd/MM/yyyy HHmmss).

Here are the Steps of UniGWS checking API Credentials:

Step	Check	Description
1	$ \text{NowTime} - \text{RequestTime} < 5 \text{ min}$	"NowTime" is time to receive response from UniGWS. Result: + true: valid + false: invalid Note: + AgentId is unique. + Based on AgentId (used as a identity), VietUnion will verify IP Addresses, certificate for authentications.

Digital Signature Authentication

If VietUnion required for using digital signature, Backend Partner would implement the requirement in this section. As calling any API of UniGWS, Backend Partner always response 2 following values:

Value	Description
RequestData	RequestData is a XML range what contains all parameters for API.
Signature	Digital signature for RequestData. Signature is set up by following rule: Signature = SIGN (Backend Partner PrivateKey, RequestData); Note: + SIGN (): using RSA with SHA256 algorithm. + Backend Partner PrivateKey: this is the private key of Partner. Corresponding to this key, Partner need to send VietUnion the public key. Vietunion base on this to authenticate the signature of Partner.

After responding to Backend Partner, any API in UniGWS always response 2 following values:

Value	Description
ResponseData	ResponseData is a XML range what contain all parameters for a API.
Signature	String type is digital signature for RequestData. Signature is set up by following rule: Signature = SIGN (VietUnion PrivateKey, ResponseData); Note: + SIGN ():using RSA with SHA256 algorithm. + VietUnion PrivateKey: this is the private key of VietUnion. Corresponding to this key, VietUnion will send back the Partner the public key. Partner base on this to authenticate the signature of Vietunion.

API List

API General Structure

UniGWS offer a unique API that generally use for all commands.

Declaration C#

```
public UniGWSResponse Execute6 (UniGWSRequest Request)
```

Input

```
<?xml version="1.0" encoding="UTF-8"?>
<UniGWSRequest>
  <Request>
    <RequestTime>string</RequestTime>
    <Operation>string</Operation>
    <RequestData>string</RequestData>
    <Signature>string</Signature>
  </Request>
</UniGWSRequest>
```

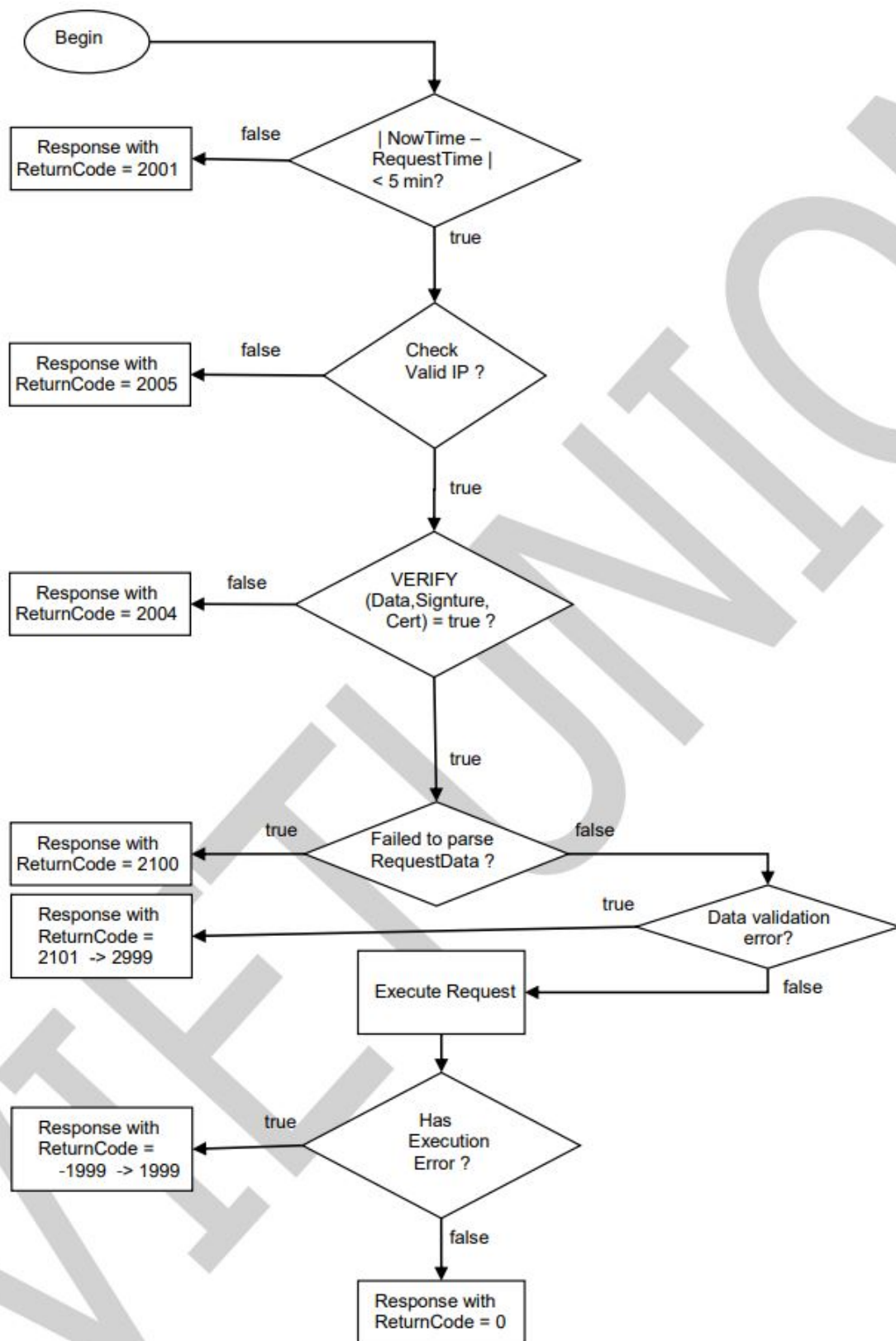
Parameter	Description
RequestTime	Time for Backend Partner call API (dd/MM/yyyy HHmmss)
Operation	Command name need to execute and apply. With each value of Operation, RequestData will setup with matched XML.
RequestData	XML string that contains all parameters of command (Operation). According to value of Operation that have rule for structure of XML in correspondence RequestData. XML details are described in each of command.
Signature	Included digital signature for RequestData. If Backend Partner were not required for using digital signature, Its value is null or empty. If digital signature is required, Signature is setup by this following rule: <code>Signature = SIGN (Backend Partner PrivateKey, RequestData);</code>

Return

```
<?xml version="1.0" encoding="UTF-8"?>
<UniWSResponse>
  <ResponseData>string</ResponseData>
  <Signature>string</Signature>
</UniWSResponse>
```

Parameter	Description
ResponseData	XML string that contains all parameters returned of the command (Operation). Depend on value of Operation that have rule for structure of XML in correspondence ResponseData. XML details are described in each of command.
Signature	XML details are described in each of command. Be digital signature for ResponseData. If Backend Partner were not required for using digital signature, Its value is null or empty. If digital signature is required, Signature is setup by this following rule: <code>Signature = SIGN (VietUnion PrivateKey, ResponseData);</code>

Execute



API Online Bill Payment

Purpose

API is used for making online bill payment.

Command code

```
operation = "MMS_PayBillForSDK"
```

RequestData

```
class PayBillBERequest
```

```
<?xml version="1.0" encoding="UTF-8"?>
<PayBillBERequest>
  <UserId>string</UserId>
  <AgentId>string</AgentId>
  <SystemTrace>string</SystemTrace>
  <TransactionTime>string</TransactionTime>
  <MoneyTotal>int</MoneyTotal>
  <BillPaymentInfo>string</BillPaymentInfo>
  <ContactPhone>string</ContactPhone>
  <ChannelPerform>string</ChannelPerform>
  <LanguageCountryCode>string</LanguageCountryCode>
</PayBillBERequest>
```

Parameter	Type	Max Length	Nullable	Description
UserId	String	50	N	User account in the system of the partner.
AgentId	String	15	N	This is the partner identity code. This value is issued by Vietunion
SystemTrace	String	50	N	This value is ID of transaction and must be unique for each transaction. It is used to check information related to transactions and reconciliation between Partner – VietUnion
TransactionTime	String	14	N	Time for executing the transaction. Format yyyyMMddHHmmss
MoneyTotal	Decimal	9	N	Total amount of bills customers want to pay.
BillPaymentInfo	String	1000	N	This is the bill information that wants to make the payment. The SDK will wrapper this information after the User click to confirm payment.
ContactPhone	String	11	N	This is the contact phone of user. Purpose of wanting to resolve complaints and supporting customers with payment related issues
ChannelPerform	String	7	N	The channel action. Including 04 value: - COUNTER => action at counter. - IB => action on the internet banking. - MOBI => action on mobi app. - AUTO => auto payment.

Parameter	Type	Max Length	Nullable	Description
LanguageCountryCode	String	5	Y	Language country code. Including values: - vi-VN => Vietnam. This is the default value. - en-GB => English.

ResponseData

```
class PayBillBEResult
```

```
<?xml version="1.0" encoding="UTF-8"?>
<PayBillBEResult>
  <ReturnCode>int</ReturnCode>
  <OrderNo>string</OrderNo>
  <DescriptionCode>string</DescriptionCode>
  <AddInfo>string</AddInfo>
  <SubReturnCode>int</SubReturnCode>
  <ExtraInfo>string</ExtraInfo>
</PayBillBEResult>
```

Parameter	Type	Max Length	Nullable	Description
ReturnCode	Int	4	N	Refer to ReturnCode
OrderNo	String	50	Y	Vietunion transaction ID.
AddInfo	String	300	N/Y	More information are returned by VietUnion. AddInfo Format: TITLE:VALUE/TITLE:VALUE , etc... The partner can use to display on the receipt or history of transaction.
DescriptionCode	String	300	N/Y	The message is attached with Error Code -9 (ReturnCode = -9).
SubReturnCode	Int	4	Y	This is the sub-error code and returned with ReturnCode = -9 (Please refer to the DescriptionError.xlsx file for details)
ExtraInfo	String	100	Y	Refer to ExtraInfo

ReturnCode

Code	Description
0	Making payment successfully. <i>In this case, the partner:</i> - Collect money of customer. - Update transaction status. - Inform to customer. The message refer "The payment transaction is successful."
1	Making payment for VietUnion successfully, but bill is still exist on the biller's system. <i>In this case, the partner:</i> - Still hold on customer's money. - Retry n time (recommend 3 times) to check transaction status. (See reference at API GetTransactionStatus) Or - Settlement later
-1	Bill Payment is failure. <i>This case, the Partner refund and inform to customer.</i> <i>The message refer "System error".</i>
-3	Bill Payment is timeout. <i>In this case, the partner:</i> - Still hold on customer's money. - Retry n time (recommend 3 times) to check transaction status. (See reference at API GetTransactionStatus) Or Settlement later.
-4	To Enter Payment amount is not within the Range (Min, Max) allow. <i>This case, the Partner refund to customer.</i> <i>The message refer "System error."</i>
-7	Bill Payment is failure. Because The deposit wallet of Partner does not have enough money to pay bill This case, the Partner refund to customer. The message refer "System error"
-9	Bill payment is failure. <i>This case, the partner:</i> - Refund to customer. - Inform to customer. The content of message is value off ReturnCodeDescription param.
2001	Bill Payment is failure (Invalid time between Partner – VietUnion) <i>This case, the Partner refund to customer.</i> <i>The message refer "System error"</i>
2004	Bill Payment is failure (Invalid signature) <i>This case, the Partner refund to customer.</i> <i>The message refer "System error"</i>
2005	Bill Payment is failure (Invalid IP) <i>This case, the Partner refund to customer.</i> <i>The message refer "System error"</i>

ExtraInfo

This is the extra information and returned with SubReturncode.

SubReturncode	Description
-119	ExtralInfo is " ORDER_CODE:[X] ". [X] is the order code. Example: ORDER_CODE: 1234566
-112	ExtralInfo is " CUSTOMER_ID:[X]/MAX_AMOUNT:[Y] ". In this, [X] is customer code and [Y] is max amount. Example: CUSTOMER_ID: 1234566/MAX_AMOUNT:2000000
-113	ExtralInfo is " CUSTOMER_ID:[X]/MAX_AMOUNT:[Y] ". In this, [X] is customer code and [Y] is max amount. Example: CUSTOMER_ID: 1234566/MAX_AMOUNT:2000000
-114	ExtralInfo is " CUSTOMER_ID:[X]/MAX_TIMES:[Z] ". In this, [X] is customer code and [Z] is max times. Example: CUSTOMER_ID: 1234566/ MAX_TIMES:4
-115	ExtralInfo is " AMOUNT:[Y] ". In this, [Y] is amount. Example: AMOUNT: 10000000
-116	ExtralInfo is " AMOUNT:[Y] ". In this, [Y] is amount. Example: AMOUNT: 20000

API Get Transaction Status

Purpose

The API is used for calling to the VietUnion system, in order to check transaction status without waiting for confirmation of the VietUnion push. **Note: This API just apply for the Bill payment and Air time service.**

Command code

```
Operation = "MMS_GetTransactionStatusBE"
```

RequestData

```
class GetTransactionStatusBERequest
```

```
<?xml version="1.0" encoding="UTF-8"?>
<GetTransactionStatusBERequest>
  <SystemTrace>string</SystemTrace>
  <RequestTime>string</RequestTime>
  <TransactionTime>string</TransactionTime>
  <AgentId>string</AgentId>
</GetTransactionStatusBERequest>
```

Parameter	Type	Max Length	Nullable	Description
AgentId	String	15	N	This is the partner identity code. This value is issued by VietUnion.
SystemTrace	String	50	N	This value is ID of transaction of partner. (refer to SystemTrace in payment request).
RequestTime	String	14	N	This is the time execute to check transaction status. Format: yyyyMMddHHmmss
TransactionTime	String	14	N	This is the time of payment transaction. Format: yyyyMMddHHmmss Refer to TransactionTime at API Online bill payment)

ResponseData

```
class GetTransactionStatusBEResult
```

```
<?xml version="1.0" encoding="UTF-8"?>
<GetTransactionStatusBEResult>
  <ReturnCode>int</ReturnCode>
  <Status>int</Status>
  <OrderNo>string</OrderNo>
</GetTransactionStatusBEResult>
```

Parameter	Type	Max Length	Nullable	Description
ReturnCode	Int	4	N	Refer to ReturnCode
Status	Int	1	N	Refer to Status
OrderNo	String	50	Y	VietUnion TransactionId

ReturnCode

Code	Description
0	Checking status successful. In this case, Partner need to refer to the value of Status property below
-1	The processing failure. <i>In this case, the partner needs to retry again.</i>
-2	Transaction is not exist. <i>In this case, the partner:</i> - Refund to customer. - Inform to customer. The message refer "The payment transaction is failed."
2001	Getting failed (Invalid time between Partner – VietUnion) <i>In this case, Partner retry</i>
2004	Getting failed (Invalid signature.) <i>In this case, Partner retry.</i>
2005	Getting failed (Invalid IP.) <i>In this case, Partner retry.</i>

Status

Code	Description
0	Transaction status is pending.
1	Transaction status is successful.
2	Transaction status is cancel
3	Transaction status is processing
4	Paid for VietUnion, but bill is still exist on the provider system.
5	Transaction status is failure.

Note:

- With value of **1, the Partner need to:**
 - Update transaction status to success.
 - Collect money of customer
 - Inform to customer
- With value of **2 or 5 the Partner need to:**
 - Update transaction status to be failed.
 - Refund to customer
 - Inform to customer
- **Other value the Partner need to:**
 - Hold on customer's money.
 - Retry again.

Contact

- Address: Floor 4th, Harbour View Tower, 35 Nguyen Hue Street, Ben Nghe Ward, District 1, Ho Chi Minh City, Vietnam.
- Tel: 08. 39117147
- Email: support@payoo.vn

RESEARCH AND DEVELOPMENT DEPT.

Mr. Nguyen Huu Nhan

- Email: nhan.nguyen@vietunion.com.vn
- Mobile: 0355964968
- Phone: (84-8) 3911 7147 (543)